

STUDY NOTES

Hugging Face LLM Course

Chapter 2 — Using Transformers

A structured reference covering the AutoModel & AutoTokenizer APIs, what happens behind pipeline(), tokenization strategies, padding & truncation, attention masks, batching, and optimized inference deployment (TGI, vLLM, llama.cpp) — with a cheat sheet, glossary, interview questions, quiz, and hands-on exercises.

Hugging Face LLM Course — Chapter 2

Using ■ Transformers — Study Notes

Concise reference notes covering the AutoModel/AutoTokenizer APIs, what pipeline() does under the hood, tokenization algorithms, padding/truncation/attention masks, batching multiple sequences, and how production LLM serving frameworks (TGI, vLLM, llama.cpp) optimize inference.

Table of Contents

1. Chapter Overview — From pipeline() to the Model API
2. Behind the Pipeline — Preprocess → Model → Postprocess
3. Models — AutoModel, Configs, and Checkpoints
4. Tokenizers — Word / Character / Subword Approaches
5. Handling Multiple Sequences — Batching, Padding, Attention Masks
6. Putting It All Together — The Full Tokenizer API
7. Optimized Inference Deployment — TGI, vLLM, llama.cpp
8. Chapter Summary
9. Cheat Sheet
10. Terminology
11. Acronyms
12. Interview Questions (20)
13. Mini Quiz — 20 MCQs
14. Hands-On Exercises
15. Revision Checklist

1. Chapter Overview — From pipeline() to the Model API

OVERVIEW

Chapter 1 introduced **pipeline()** as a one-line way to run inference. Chapter 2 opens the hood: it replicates `pipeline()` by hand using the **Transformers** **AutoTokenizer** and **AutoModel** classes, then works through the model API, the tokenizer API, batching, and finally how these pieces are packaged for production-grade serving.

WHY IT MATTERS

Every later chapter (fine-tuning, sharing models, datasets) assumes you're comfortable manually wiring a tokenizer to a model. This chapter is the bridge between "pipeline() just works" and "I understand why it works, and can customize it."

CORE CONCEPTS

- The chapter's spine mirrors the pipeline's three stages: **tokenizer** (text → numbers), **model** (numbers → logits), **postprocessing** (logits → labels) — each gets its own deep-dive section.
- **Transformers** models are, under the hood, just PyTorch `nn.Module` objects — nothing magic, fully inspectable and hackable.
- The library favors **simplicity over sharing**: each model has its own layers rather than reusing modules across files, trading some code duplication for easier standalone understanding.
- The chapter closes by zooming out from "a model on my laptop" to "a model serving thousands of requests," introducing TGI, vLLM, and llama.cpp.

KEY TAKEAWAY

Chapter 1 taught *what* Transformers can do; Chapter 2 teaches *how* the library is actually structured underneath `pipeline()` — tokenizer object, model object, config object — so you can swap, inspect, and combine them yourself instead of only calling a wrapper.

EXAMPLE

Prototyping in Chapter 1 with `pipeline("sentiment-analysis")` is fine for a demo; but building a batching-aware, GPU-efficient microservice requires knowing exactly what tensors go in and come out — which is exactly what this chapter unpacks.

■ **DEVELOPER TIP** — *Whenever you're debugging a `pipeline()` result that looks wrong, dropping down to the manual `tokenizer` → `model` → `postprocess` steps from this chapter is the fastest way to isolate which stage is misbehaving.*

2. Behind the Pipeline — Preprocess → Model → Postprocess

OVERVIEW

This section reproduces, step by step and by hand, exactly what pipeline("sentiment-analysis") does internally: tokenize the raw text, run it through the base Transformer plus a task head, then convert raw logits into human-readable labels.

WHY IT MATTERS

Once this three-stage flow clicks as concrete objects (a tokenizer object, a model object, a softmax call) instead of an abstract idea, every other pipeline task becomes "the same shape" with a different head and a different postprocessing function.

CORE CONCEPTS

- **Preprocessing:** `AutoTokenizer.from_pretrained(checkpoint)` downloads/caches the exact tokenizer used when the model was pretrained, then returns a dict with `input_ids` and `attention_mask` once called on raw text.
- Transformer models only accept **tensors**, not plain lists — pass `return_tensors="pt"` (or `"np"`) to get framework-ready output instead of a list of lists.
- **Going through the model:** `AutoModel.from_pretrained(checkpoint)` loads only the base Transformer body. Its output is a **hidden state** tensor of shape (*batch size, sequence length, hidden size*) — e.g. (2, 16, 768) — a high-dimensional, not-yet-useful contextual representation.
- **Model heads:** a small set of extra layers sits on top of the base model and projects hidden states into a task-specific shape. Swapping `AutoModel` for `AutoModelForSequenceClassification` attaches a classification head, shrinking the output to (batch size, num labels) — e.g. (2, 2).
- **Postprocessing:** raw model output is **logits** — unnormalized scores, not probabilities. A softmax layer converts logits into probabilities; `model.config.id2label` maps output positions to human-readable label names.

KEY TAKEAWAY

`pipeline()` is not doing anything you couldn't do yourself: *tokenizer → base model → head → softmax → id2label* is the entire trick. Logits are not probabilities — always softmax before reading them as confidence scores.

EXAMPLE

For "I've been waiting for a HuggingFace course my whole life.", the head outputs logits like [-1.56, 1.61]; softmax turns that into [0.04, 0.96], and `id2label = {0: 'NEGATIVE', 1: 'POSITIVE'}` turns that into POSITIVE at 0.96 confidence.

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification
import torch

checkpoint = "distilbert-base-uncased-finetuned-sst-2-english"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
model = AutoModelForSequenceClassification.from_pretrained(checkpoint)

inputs = tokenizer(["I loved this course!"], padding=True, return_tensors="pt")
outputs = model(**inputs)
probs = torch.nn.functional.softmax(outputs.logits, dim=-1)
# probs -> [[0.0402, 0.9598]] (NEGATIVE, POSITIVE)
```

■ *DEVELOPER TIP* — `model.config.id2label` is the fastest way to check what a classification checkpoint's output positions actually mean — never assume position 0 is always "negative".

■ **COMMON MISTAKE** — Reading raw logits as confidence scores directly (e.g. treating 4.17 as "a high probability") instead of passing them through softmax first.

3. Models — AutoModel, Configs, and Checkpoints

OVERVIEW

This section covers the model side of the API in isolation: how to instantiate a Transformer from a Hub checkpoint, what gets saved to disk, and how raw text becomes the `input_ids` / `token_type_ids` / `attention_mask` dictionary a model consumes.

WHY IT MATTERS

Knowing exactly what `from_pretrained()` downloads and what `save_pretrained()` writes demystifies the Hub — a checkpoint is just two files, not a black box.

CORE CONCEPTS

- `AutoModel.from_pretrained("bert-base-cased")` is a thin, "auto" wrapper that guesses and instantiates the right architecture class for a checkpoint; the equivalent explicit call is `BertModel.from_pretrained(...)` if the architecture is already known.
- `save_pretrained("dir")` writes exactly two files: **config.json** (architecture + metadata) and a weights file (e.g. **model.safetensors**) — config defines the shape, weights are the learned numbers.
- `push_to_hub("name")` uploads both files to a namespaced Hub repo so anyone can later run `AutoModel.from_pretrained("username/name")`.
- **Encoding text:** calling a tokenizer directly returns a dict with three keys: `input_ids` (numeric token IDs), `token_type_ids` (which sentence — A or B — each token belongs to), and `attention_mask` (which tokens are real vs. padding).
- **Special tokens** like BERT's [CLS] and [SEP] are inserted automatically by the tokenizer, because the model was pretrained expecting them — not every model needs them, and some add them only at the start or only at the end.
- **Padding** (`padding=True`) and **truncation** (`truncation=True`, optionally with `max_length`) are both tokenizer arguments that make a batch of variable-length sentences fit into one rectangular tensor.

KEY TAKEAWAY

A "model" on the Hub is really a config (architecture blueprint) plus a weights file (learned parameters) — `from_pretrained()` and `save_pretrained()` are the two verbs that move a model between disk, memory, and the Hub.

EXAMPLE

Loading `bert-base-cased` gives a 12-layer, 768-hidden-size, 12-attention-head BERT model with case-sensitive tokenization — inspecting its `config.json` shows those exact numbers.

```
from transformers import AutoTokenizer, AutoModel

tokenizer = AutoTokenizer.from_pretrained("bert-base-cased")
model = AutoModel.from_pretrained("bert-base-cased")

encoded = tokenizer("Hello, I am a single sentence!")
# {'input_ids': [...], 'token_type_ids': [...], 'attention_mask': [...]}
tokenizer.decode(encoded["input_ids"])
# "[CLS] Hello, I am a single sentence! [SEP]"
```

■ **COMMON MISTAKE** — Forgetting that padding tokens get input ID 0 and `attention_mask` 0 by convention for many checkpoints — always check `tokenizer.pad_token_id` rather than assuming 0 for every model.

4. Tokenizers — Word / Character / Subword Approaches

OVERVIEW

Tokenizers exist purely to translate raw text into numbers a model can process, and back again. This section compares the three families of tokenization algorithm and walks through the concrete encode → convert-to-IDs → decode pipeline.

WHY IT MATTERS

The tokenization strategy a model was pretrained with directly shapes vocabulary size, how well it handles rare/unseen words, and how many tokens a given sentence costs — all of which affect both accuracy and inference cost.

CORE CONCEPTS

- **Word-based:** split on whitespace/punctuation. Simple and decent results, but needs a huge vocabulary (500,000+ words for English) and treats related forms like "dog" and "dogs" as totally unrelated IDs. Requires an [UNK] token for anything unseen.
- **Character-based:** split into individual characters. Tiny vocabulary and almost no unknown tokens, but each character carries little standalone meaning and sequences balloon in length (one word → 10+ tokens).
- **Subword tokenization (the modern default):** keep frequent words whole; split rare words into meaningful pieces ("annoyingly" → "annoying" + "ly"). Gets small vocabularies, near-zero unknown tokens, and preserved semantic meaning — especially strong for agglutinative languages like Turkish.
- Named subword variants to recognize: **Byte-level BPE** (GPT-2), **WordPiece** (BERT), **SentencePiece / Unigram** (many multilingual models).
- Loading/saving a tokenizer mirrors models exactly: `from_pretrained()` / `save_pretrained()`, and `AutoTokenizer` auto-selects the correct tokenizer class for any checkpoint name.
- **Encoding = `tokenizer.tokenize()` + `convert_tokens_to_ids()`.** **Decoding** reverses IDs back to a readable string, and intelligently re-merges subwords into whole words (crucial for generation/translation output).

KEY TAKEAWAY

Subword tokenization is the sweet spot between word-based (huge vocab, brittle) and character-based (tiny vocab, long sequences, low per-token meaning) — it's why BERT splits "transformer" into "transform" + "##er" instead of treating it as one unknown word or eight separate characters.

EXAMPLE

Tokenizing "Using a Transformer network is simple" with BERT's tokenizer yields ['Using', 'a', 'transform', '##er', 'network', 'is', 'simple'] — six of seven words stay whole; only the rarer "Transformer" splits into two meaningful subword pieces.

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-cased")
tokens = tokenizer.tokenize("Using a Transformer network is simple")
# ['Using', 'a', 'transform', '##er', 'network', 'is', 'simple']

ids = tokenizer.convert_tokens_to_ids(tokens)
decoded = tokenizer.decode(ids)
# 'Using a Transformer network is simple'
```

■ *DEVELOPER TIP — A lot of "unknown token" or garbled-output bugs in production trace back to using a tokenizer that doesn't match the model checkpoint — always load them from the same checkpoint name together.*

5. Handling Multiple Sequences — Batching, Padding, Attention Masks

OVERVIEW

Real workloads rarely process one short sentence at a time. This section explains why models expect a **batch** dimension even for a single sentence, why variable-length sequences need padding, and why padding alone silently corrupts results unless paired with an attention mask.

WHY IT MATTERS

This is the single most common source of "my manual code doesn't match pipeline() output" bugs — skipping the batch dimension or the attention mask produces either a crash or silently wrong logits.

CORE CONCEPTS

- Transformers models expect a **batch of sequences** by default, shaped *(batch size, sequence length)* — passing a single 1D tensor of IDs raises an `IndexError`; wrap it as `[ids]` or call the tokenizer directly, which adds the batch dimension automatically.
- **Batching** = sending multiple sequences through the model in one forward pass. Even a single sentence is technically "a batch of one."
- **Padding** makes a batch of different-length sequences rectangular by appending a special padding token (ID from `tokenizer.pad_token_id`) to shorter sequences until all match the longest one.
- Padding alone is **not safe**: because attention lets every token see every other token, padding tokens would otherwise influence the real tokens' representations and change the output.
- **Attention masks** are a same-shaped tensor of 1s (attend) and 0s (ignore) passed alongside `input_ids`; they tell attention layers to skip padding tokens entirely, restoring identical results to running each sentence individually.
- **Sequence length limits**: most models cap out around 512–1024 tokens and will error past that. Fix options: use a long-context architecture (e.g. Longformer, LED) or truncate the input.

KEY TAKEAWAY

Batching + padding + an attention mask is one package deal, not three optional features — omit the mask and padded batches silently return different logits than the same sentences run one at a time.

EXAMPLE

Sentence A (3 tokens) and Sentence B (2 tokens) padded together to length 3 will give the *wrong* logits for B unless the attention mask `[1,1,0]` is passed to tell the model to ignore the padding slot.

```
batched_ids = [
    [200, 200, 200],
    [200, 200, tokenizer.pad_token_id],
]
attention_mask = [
    [1, 1, 1],
    [1, 1, 0],
]

outputs = model(
    torch.tensor(batched_ids),
    attention_mask=torch.tensor(attention_mask),
```

```
)  
# logits for row 2 now match running sentence B alone
```

■ **COMMON MISTAKE** — Padding a batch and passing it straight to the model without also passing `attention_mask` — this runs without error but silently returns wrong logits for the padded sentences.

6. Putting It All Together — The Full Tokenizer API

OVERVIEW

Sections 3–5 did tokenization, padding, truncation, and tensor conversion by hand to build intuition. This section shows that the high-level `tokenizer(...)` call already does *all* of that in a single line — for one sentence, many sentences, with any padding strategy, any truncation rule, and any output tensor type.

WHY IT MATTERS

This is the "you don't actually need to do it manually" payoff section — it consolidates everything into the one call you'll use in real code from here on.

CORE CONCEPTS

- One call handles single sentences, lists of sentences, padding, truncation, and tensor conversion together: `tokenizer(sequences, padding=True, truncation=True, return_tensors="pt")`.
- Padding strategies: `padding="longest"` (pad to the longest sequence in the batch), `padding="max_length"` (pad to the model's own max length, e.g. 512 for BERT), or `padding="max_length", max_length=N` (pad to a custom length).
- Truncation: `truncation=True` caps sequences at the model's max length; add `max_length=N` to cap at a custom shorter length instead.
- `return_tensors` selects the framework: "pt" for PyTorch tensors, "np" for NumPy arrays.
- The tokenizer automatically inserts the special tokens (like [CLS] / [SEP]) the model was pretrained with — comparing `tokenizer(sequence)` output to manual `tokenize()` + `convert_tokens_to_ids()` output shows exactly which IDs got added.

KEY TAKEAWAY

In practice you almost never call `tokenize()`, `convert_tokens_to_ids()`, and manual padding separately — you call the `tokenizer` object directly with the right keyword arguments, and it returns a model-ready dict in one shot.

EXAMPLE

The same `tokenizer(sequences, padding=True, truncation=True, return_tensors="pt")` call replaces roughly ten lines of manual `tokenize/pad/mask/tensor-convert` code from Section 5.

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification

checkpoint = "distilbert-base-uncased-finetuned-sst-2-english"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)
model = AutoModelForSequenceClassification.from_pretrained(checkpoint)

sequences = [
    "I've been waiting for a HuggingFace course my whole life.",
    "So have I!",
]
tokens = tokenizer(sequences, padding=True, truncation=True, return_tensors="pt")
output = model(**tokens)
```

■ **DEVELOPER TIP** — Always prefer calling the `tokenizer` object directly over chaining `tokenize()` + `convert_tokens_to_ids()` by hand in production code — it's shorter, less error-prone, and adds special tokens correctly for you.

7. Optimized Inference Deployment — TGI, vLLM, llama.cpp

OVERVIEW

The final technical section zooms out from "run one forward pass locally" to "serve an LLM to many concurrent users in production," introducing three purpose-built serving frameworks and the core optimization techniques behind them.

WHY IT MATTERS

A model that runs fine in a notebook can be far too slow or memory-hungry to serve at scale; these frameworks exist specifically to close that gap without changing the model's outputs.

CORE CONCEPTS

- **Flash Attention:** optimizes the attention mechanism's memory movement rather than its math — instead of repeatedly shuttling data between slow high-bandwidth memory (HBM) and fast on-chip SRAM, it loads data into SRAM once and computes there, cutting the memory-transfer bottleneck that leaves GPUs idle.
- **vLLM / PagedAttention:** tackles KV-cache memory management by splitting the cache into fixed-size "pages" (like OS virtual memory) that don't need to sit contiguously in GPU memory and can be shared across sequences — reported to reach up to 24x higher throughput than naive KV-cache handling.
- **Text Generation Inference (TGI):** Hugging Face's own production server, built for stable, predictable memory usage via fixed sequence lengths, Flash Attention 2, and continuous batching.
- **llama.cpp:** a highly optimized C/C++ implementation built for CPU efficiency (with optional GPU acceleration), ideal for resource-constrained environments; relies heavily on **quantization** to shrink models.
- **Quantization** lowers weight precision (e.g. 32-bit/16-bit float down to 8-bit or 4-bit integers), trading a small amount of accuracy for large reductions in memory footprint and, on constrained hardware, faster inference.

KEY TAKEAWAY

Model architecture answers "what can this model do"; the serving framework answers "how many people can use it at once, and how cheaply" — Flash Attention and PagedAttention are both about using GPU memory smarter, not about changing what the model computes.

EXAMPLE

A team serving a chat LLM to thousands of concurrent users would reach for TGI or vLLM on GPU hardware; a team shipping a local, offline assistant on a laptop or edge device would reach for llama.cpp with a quantized model instead.

Framework	Best for	Key technique
TGI	Stable, predictable production serving, deep HF integration	Flash Attention 2 + continuous batching
vLLM	Maximizing throughput under many concurrent requests	PagedAttention (paged KV-cache)
llama.cpp	CPU-only / resource-constrained, edge, offline use	Quantization (8-bit/4-bit weights)

■ **DEVELOPER TIP** — If GPU memory isn't the bottleneck for your use case, quantization can still add a small latency cost from the extra quantize/dequantize step — benchmark before assuming it's a free win.

8. Chapter Summary

The whole chapter in five lines

- `pipeline()` is just *tokenizer* → *base model* → *task head* → *postprocess*, wired together — every piece is independently accessible via `AutoTokenizer` / `AutoModel` / `AutoModelForX`.
- A checkpoint is two files: a config (architecture blueprint) and a weights file (learned numbers) — `from_pretrained()/save_pretrained()/push_to_hub()` move it between disk, memory, and the Hub.
- Subword tokenization is the modern default because it balances vocabulary size, unknown-token rate, and preserved meaning better than pure word- or character-based approaches.
- Models expect a batch dimension; padding makes variable-length batches rectangular, but padding is only correct when paired with an attention mask that tells the model which tokens to ignore.
- The high-level `tokenizer(...)` call already does `tokenize` + `pad` + `truncate` + `tensor-convert` in one line — and production serving frameworks (TGI, vLLM, llama.cpp) apply memory- and cache-level optimizations on top of that to scale inference.

What clicks

- Hidden states are the raw, general-purpose Transformer output; a task head is what turns them into something task-specific like class logits.
- Padding without an attention mask is a silent-failure bug, not a crash — always pair them.
- "Optimizing inference" is mostly about memory movement (Flash Attention, PagedAttention) and reduced precision (quantization), not a different model.

Worth digging into further

- Fine-tuning a pretrained checkpoint on a custom dataset — covered in Chapter 3.
- The ■ Datasets and ■ Tokenizers libraries in more depth — Chapters 5–6.
- KV-caching mechanics and speculative decoding for generation speed, referenced but not detailed here.

9. Cheat Sheet

NEED TO...	REACH FOR...
Load a pretrained model	<code>AutoModel.from_pretrained(checkpoint)</code>
Load a model with a task head	<code>AutoModelForSequenceClassification</code> / ... <code>ForCausalLM</code> / ... <code>ForQuestionAnswering</code>
Load the matching tokenizer	<code>AutoTokenizer.from_pretrained(checkpoint)</code> — same checkpoint name
Turn logits into probabilities	<code>torch.nn.functional.softmax(logits, dim=-1)</code>
Map output index to a label	<code>model.config.id2label</code>
Tokenize + pad + truncate + tensorize in one call	<code>tokenizer(seqs, padding=True, truncation=True, return_tensors="pt")</code>
Fix a batch of different-length sequences	<code>padding=True</code> *and* pass <code>attention_mask</code>
Handle sequences longer than the model supports	<code>truncation=True</code> , or a long-context model (Longformer/LED)
Serve an LLM to many concurrent users on GPU	TGI or vLLM (PagedAttention)
Run a model on CPU / edge hardware	llama.cpp with a quantized (8-bit/4-bit) checkpoint

Three-line mental model

```
text --tokenizer--> input_ids + attention_mask
input_ids --base model--> hidden states (batch, seq_len, hidden_size)
hidden states --task head + postprocess--> labels / generated text
```

Decision table

Question to ask	Answer tells you
Do sentences in my batch have different lengths?	You need <code>padding=True</code> + an <code>attention_mask</code>
Is my sequence longer than the model's max length?	<code>truncation=True</code> , or switch to a long-context model
Do I need class labels, not raw vectors?	Use an <code>AutoModelFor*</code> task head, not plain <code>AutoModel</code>
Am I deploying to many concurrent GPU users?	Look at TGI or vLLM
Am I deploying to CPU / a resource-constrained device?	Look at llama.cpp + quantization

10. Terminology

Term	Working definition
Checkpoint	A specific saved set of pretrained weights + config, identified by a Hub name (e.g. bert-base-cased).
AutoModel	A wrapper class that auto-detects and instantiates the correct model architecture for a given checkpoint.
Hidden state	The high-dimensional vector output by the base Transformer body, before any task-specific head is applied.
Model head	One or a few extra layers on top of the base model that project hidden states into a task-specific output shape.
Logits	The raw, unnormalized scores output by a model's final layer, before softmax is applied.
Softmax	A function that converts raw logits into a probability distribution that sums to 1.
id2label	A config mapping from a model's output index (e.g. 0, 1) to a human-readable label name.
Word-based tokenizer	Splits text on whitespace/punctuation into whole words; simple but produces huge vocabularies.
Character-based tokenizer	Splits text into individual characters; tiny vocabulary, very long sequences.
Subword tokenizer	Keeps common words whole and splits rare words into meaningful pieces; the modern default (BPE, WordPiece, etc.).
Batching	Sending multiple sequences through the model together in one forward pass.
Padding	Adding a special padding token to shorter sequences so a batch becomes a single rectangular tensor.
Attention mask	A 0/1 tensor telling attention layers which tokens are real (1) versus padding to ignore (0).
Truncation	Cutting a sequence down to a maximum token length the model can handle.
Special tokens	Model-specific markers like [CLS]/[SEP] that the tokenizer inserts automatically because the model expects them.
Flash Attention	An attention implementation optimized for GPU memory movement, not attention's underlying math.
PagedAttention	vLLM's technique of splitting the KV cache into non-contiguous, shareable "pages" for higher throughput.
Quantization	Reducing weight precision (e.g. to 8-bit or 4-bit) to shrink memory footprint, used heavily by llama.cpp.

11. Acronyms

Acronym	Stands for
API	Application Programming Interface
BPE	Byte Pair Encoding
CLS	(Not strictly an acronym) — the classification special token in BERT-style models
SEP	(Not strictly an acronym) — the separator special token in BERT-style models
UNK	Unknown (token) — placeholder for out-of-vocabulary words
KV cache	Key-Value cache — stored attention keys/values reused across generation steps
HBM	High Bandwidth Memory — a GPU's main (slower) memory
SRAM	Static Random-Access Memory — a GPU's fast on-chip cache used by Flash Attention
TGI	Text Generation Inference — Hugging Face's production LLM serving framework
vLLM	(Library name, not a strict acronym) — high-throughput LLM serving library using PagedAttention
GPU	Graphics Processing Unit
INT8	8-bit Integer (a common quantized weight precision)

12. Interview Questions (20)

1. What three stages does `pipeline()` perform under the hood?

Preprocessing with a tokenizer (text → input IDs + attention mask), a forward pass through the model, and postprocessing (e.g. softmax + id2label) to produce a readable result.

2. What is the difference between `AutoModel` and `AutoModelForSequenceClassification`?

`AutoModel` loads only the base Transformer body and returns hidden states; `AutoModelForSequenceClassification` adds a classification head on top, returning class logits instead.

3. What shape does the base Transformer's output have, and what do the three dimensions mean?

(batch size, sequence length, hidden size) — number of sequences processed together, tokens per sequence, and the dimensionality of each token's contextual vector.

4. Why are logits not probabilities?

Logits are raw, unnormalized scores from the model's final layer; they need to be passed through a softmax function to become a valid probability distribution.

5. What two files does `save_pretrained()` write for a model, and what does each contain?

`config.json` (architecture and metadata) and a weights file such as `model.safetensors` (the learned parameter values); the two work together to reconstruct the model.

6. What's the core tradeoff between word-based and character-based tokenization?

Word-based needs a huge vocabulary and can't relate similar word forms, but keeps meaningful units; character-based has a tiny vocabulary and almost no unknowns, but each token carries little meaning and sequences get much longer.

7. Why did subword tokenization become the standard approach?

It keeps frequent words whole and only splits rare words into meaningful pieces, giving good vocabulary coverage with far fewer unknown tokens than word-based tokenizers and shorter sequences than character-based ones.

8. Why do **Transformers** models require a batch dimension even for one sentence?

The library's models are built to expect input shaped (batch size, sequence length); passing a single unbatched 1D tensor causes a dimension error, so even one sentence must be wrapped as a batch of size one.

9. What problem does padding solve, and what new problem does it introduce?

Padding makes variable-length sequences rectangular so they can form one tensor; but because attention lets tokens see each other, padding tokens can silently distort the real tokens' representations unless masked.

10. What does an attention mask do, concretely?

It's a same-shaped 0/1 tensor passed alongside input IDs, telling attention layers to ignore padding tokens (0) and only attend to real tokens (1), restoring the same output as running sequences unpadded.

11. What are the two ways to handle a sequence longer than a model's maximum length?

Truncate the sequence to fit, or switch to a model architecture built for longer sequences, such as Longformer or LED.

12. What does calling `tokenizer(sequences, padding=True, truncation=True, return_tensors="pt")` actually do?

It tokenizes one or many sequences, adds required special tokens, pads them to a consistent length, truncates anything too long, and returns the result as ready-to-use PyTorch tensors — all in one call.

13. Why must a tokenizer and model always come from the same checkpoint?

The tokenizer's vocabulary and special-token rules were fixed during that model's pretraining; a mismatched tokenizer produces different token IDs than the model was trained on, corrupting results.

14. What is Flash Attention optimizing, and what is it not changing?

It optimizes how attention moves data between slow GPU memory (HBM) and fast on-chip cache (SRAM), reducing idle GPU time; it does not change attention's mathematical output.

15. How does vLLM's PagedAttention improve throughput?

It splits the KV cache into fixed-size, non-contiguous pages (like OS virtual memory) that can be allocated flexibly and shared across sequences, avoiding the waste of treating the KV cache as one large contiguous block.

16. What is TGI optimized for, compared to vLLM?

TGI prioritizes stable, predictable production behavior via fixed sequence lengths, Flash Attention 2, and continuous batching, with deep Hugging Face ecosystem integration.

17. What is llama.cpp best suited for, and what technique does it rely on?

CPU-focused, resource-constrained or edge deployment; it relies heavily on quantization to shrink model size and memory usage.

18. What does quantization trade off?

It reduces weight precision (e.g. to 8-bit or 4-bit) to cut memory usage and often improve speed on constrained hardware, at the cost of some accuracy and, if GPU memory isn't a constraint, potentially a small extra latency from quantize/dequantize steps.

19. What's the difference between token_type_ids and attention_mask?

token_type_ids indicate which of two sentences (A or B) a token belongs to for sentence-pair tasks; attention_mask indicates which tokens are real content versus padding to ignore.

20. Why does the tokenizer add special tokens like [CLS] and [SEP] automatically?

Because the model was pretrained expecting them at specific positions; adding them automatically ensures inference matches the conditions the model actually learned under.

13. Mini Quiz — 20 MCQs

1. `pipeline()` internally performs which three stages?

- A. Train, validate, test
- B. Tokenize, model forward pass, postprocess
- C. Download, cache, delete
- D. Encode, compress, decode

Answer: B

2. `AutoModel` (without a task suffix) returns:

- A. Class labels
- B. Raw hidden states from the base Transformer
- C. A tokenizer object
- D. Generated text

Answer: B

3. The base Transformer output shape (batch, seq_len, hidden) — hidden size is typically:

- A. Always 2
- B. Always 512
- C. 768 or more depending on model size
- D. Not defined

Answer: C

4. Logits become probabilities after applying:

- A. ReLU
- B. Softmax
- C. Dropout
- D. Layer normalization

Answer: B

5. `model.config.id2label` is used to:

- A. Load the tokenizer
- B. Map output positions to label names
- C. Set the learning rate
- D. Save the model to disk

Answer: B

6. `save_pretrained()` on a model writes:

- A. Only the weights
- B. Only the config
- C. Both a config file and a weights file
- D. A zipped training log

Answer: C

7. Which tokenizer type needs an [UNK] token most often?

- A. Subword
- B. Character-based
- C. Word-based
- D. None of them

Answer: C

8. Character-based tokenizers mainly struggle with:

- A. Vocabulary size
- B. Sequence length ballooning and low per-token meaning
- C. Cannot handle Chinese
- D. Cannot be saved to disk

Answer: B

9. Subword tokenization splits "annoyingly" into "annoying" + "ly" because:

- A. It's alphabetical
- B. Rare words are decomposed into more frequent meaningful subwords
- C. It's a fixed rule for all words
- D. The tokenizer ran out of vocabulary

Answer: B

10. Passing a single unbatched 1D tensor of IDs to a ■ Transformers model typically:

- A. Works fine always
- B. Raises a dimension error because a batch dimension is expected
- C. Automatically pads
- D. Silently returns zeros

Answer: B

11. Padding without an attention mask:

- A. Always crashes
- B. Is completely safe
- C. Can silently change the logits for padded sequences
- D. Is not possible in ■ Transformers

Answer: C

12. An attention mask value of 0 means:

- A. Attend strongly
- B. This token should be ignored by attention
- C. This is the [CLS] token
- D. This is a rare token

Answer: B

13. If a sequence exceeds a model's max length, valid fixes include:

- A. Ignore the limit
- B. Truncate it or use a long-context model

- C. Pad it further
- D. Convert to characters

Answer: B

14. tokenizer(sequences, padding=True, truncation=True, return_tensors="pt") returns:

- A. A plain Python string
- B. A model-ready dict of PyTorch tensors
- C. Only token strings
- D. A trained model

Answer: B

15. Special tokens like [CLS] and [SEP] are added:

- A. Manually by the user every time
- B. Automatically by the tokenizer, matching pretraining
- C. Only in TensorFlow
- D. Never for BERT-based models

Answer: B

16. Flash Attention primarily optimizes:

- A. The math of attention itself
- B. Memory movement between HBM and SRAM
- C. Tokenizer vocabulary size
- D. The number of attention heads

Answer: B

17. PagedAttention (vLLM) improves throughput mainly by:

- A. Deleting the KV cache
- B. Splitting the KV cache into flexible, shareable pages
- C. Reducing model layers
- D. Removing the attention mask

Answer: B

18. TGI is designed primarily for:

- A. Local single-user notebooks only
- B. Stable, predictable production serving
- C. Training models from scratch
- D. Tokenizer visualization

Answer: B

19. llama.cpp is best suited for:

- A. Massive multi-GPU clusters only
- B. CPU-focused, resource-constrained deployment
- C. Training billion-parameter models
- D. Replacing tokenizers entirely

Answer: B

20. Quantization mainly trades off:

- A. Accuracy for reduced memory usage
- B. Vocabulary for sequence length
- C. Batch size for attention heads
- D. Nothing — it's always free

Answer: A

14. Hands-On Exercises

Exercise 1 — Reproduce pipeline() by Hand

Pick any sentiment-analysis checkpoint. Without calling `pipeline()`, load its `AutoTokenizer` and `AutoModelForSequenceClassification`, run a sentence through both, apply `softmax`, and map the result through `id2label`. Confirm it matches `pipeline("sentiment-analysis")` on the same sentence.

Exercise 2 — Tokenizer Family Comparison

Take the word "internationalization". By hand, sketch how a word-based tokenizer, a character-based tokenizer, and a subword tokenizer would each split it, and count the resulting tokens for each approach.

Exercise 3 — Padding Without a Mask

Batch two sentences of different lengths with `padding=True` but do NOT pass an `attention_mask`. Compare the logits for the shorter sentence to what you get running it alone. Explain the difference in one sentence.

Exercise 4 — Truncation Judgment Call

Given a document-summarization task where inputs average 2,000 tokens and your model's max length is 512, write two sentences on the tradeoffs of truncating versus switching to a long-context model like Longformer.

Exercise 5 — Serving Framework Fit

For each scenario, name the best-fit serving approach (TGI, vLLM, or llama.cpp) in one sentence: (1) a customer support chatbot serving 500 concurrent users on cloud GPUs, (2) an offline note-taking app running a small LLM on a user's laptop, (3) a startup needing maximum request throughput per GPU dollar.

Exercise 6 — Config Inspection

Load any checkpoint's `config.json` (via `model.config`) and identify: number of hidden layers, hidden size, number of attention heads, and vocabulary size. Explain what each number roughly controls.

15. Revision Checklist

- Explain, in one sentence, what `pipeline()` does under the hood.
- Name the difference between `AutoModel` and `AutoModelForSequenceClassification`.
- State the three dimensions of a base Transformer's hidden-state output.
- Explain why logits must be softmaxed before being read as probabilities.
- Name the two files `save_pretrained()` writes and what each contains.
- Compare word-based, character-based, and subword tokenization in one sentence each.
- Explain why models expect a batch dimension even for a single sentence.
- Explain why padding requires an attention mask to be safe.
- Describe what truncation does and when you'd choose a long-context model instead.
- Write the one-line tokenizer call that tokenizes, pads, truncates, and returns tensors together.
- Explain what Flash Attention optimizes (and what it doesn't change).
- Explain how PagedAttention improves throughput in vLLM.
- Compare TGI, vLLM, and llama.cpp by their best-fit use case.
- Explain what quantization trades off.
- Name at least 10 terms from the glossary from memory.
- Score at least 16/20 on the mini quiz without looking back.
- Re-do the mini quiz cold after a couple of weeks — a lower score is a signal to revisit this notebook.

Reference: Hugging Face LLM Course — Chapter 2 (huggingface.co/learn/llm-course/chapter2)